



The State of VBA in OpenOffice Calc

I keep trying to file my income tax returns online. However, I insist on using OpenOffice and so I have failed to accomplish what I set out to do. A request on the income tax office's site for an OpenOffice version did not get any response. A discussion on the ILUG-Delhi list pointed to a version of the ITR1 utility at <http://freedom-matters.in/>. This Excel utility was for the previous year. Changing the Excel macro code from VBA to OpenOffice Basic is time consuming. It is also hard to keep up with updated releases, and the various versions of the Excel utilities for various IT forms (ITR1 through ITR4, at present).

VBA was originally included in the <http://go-oo.org/> version of OpenOffice, but was not a complete implementation. The VBA interoperability project is now a joint project of Novell and Sun (Oracle) and is active at <http://wiki.services.openoffice.org/wiki/VBA>. Hence, it was worth exploring the state of this project, as far as the utilities for various ITR forms were concerned. The objective was to experiment and see whether we could use the Excel utilities next year, and ascertain the skill levels needed.

I started with the development snapshot from <http://download.openoffice.org/next/> on Fedora. Arch Linux includes the beta and the development versions in its repositories, making it very easy to work with all three versions (stable, beta and development) of OpenOffice concurrently. The VBA module is included in the current releases of OpenOffice; however, the security options for macros have to be set to 'Medium'. In addition, in the *Load/Save* options for VBA, 'Executable code' has to be enabled for Excel sheets.

The development version I used was OOO300_m86 (build 9518). There has been considerable progress. Validation is triggered by a call to *Worksheet_Change*. OpenOffice introduces an additional macro—*Worksheet_Change_OnChange_Proxy*—in the code, which calls the *Worksheet_Change* macro. Currently, the implementation was incomplete, so the validation code failed. However, most of the validations are now carried out by hidden functions in the spreadsheet, and they are executed. To make some progress, I commented the calls to the *Worksheet_Change* macro in the proxy. With this change, while the validation was incomplete, the worksheet could be filled.

Validations for two fields—PIN code and telephone number—still failed. The solution was to set the type for these two cells as text. Execution of various macros still failed. The issue was that the cell names in *sheet1* (which was actually the second sheet) had an additional suffix '_2', that is *sheet1.PAN_2*, while the macro was trying to use *sheet1.PAN*. I filed a bug report, but meanwhile, a small Python macro helped change the names to what was expected. (It turned out I needed to add additional names for the same cells, as some hidden formulae were affected. More details are at <http://sethanil.blogspot.com/2010/08/python-macro.html>.) However, the need for this macro was short-lived.

I checked the beta version OOO300m5 (build 9521). On this version, the name conversion was done properly. Hence, I did the

rest of the experimentation on this version. The *Worksheet_Change* function is not called in this release—but, as discussed earlier, its implementation was still incomplete in the development version.

There are a number of hidden sheets. In order to debug the code and get a better idea of what was happening, it was useful to make all the hidden sheets visible. It is quite simple, as the following Python macro illustrates:

```
def unhide():
    model = XSCRIPTCONTEXT.getDocument()
    sheets = model.getSheets()
    for n in range(sheets.getCount()):
        sheet = sheets.getByIndex(n)
        sheet.IsVisible = True
```

Once I entered the data and computed the tax, the result shown was incorrect. Since the Calculator sheet was now visible, I noticed a problem—the tax was not shown properly. The problem was that the Calculator sheet was protected, but with no password. It's possible that the translation of the function from Excel to OpenOffice was in error. The code expects the password to be the value in cell 'BE1' on *Sheet6* (the Home sheet), appended with '(21)*'. The content of the cell was a null string. Hence, the password expected was '(21)*'. Manually changing the blank password to the expected password resulted in the taxes being computed correctly.

One problem I encountered was that *Not var = ""* was interpreted as (*Not var*) = "" instead of *Not (var = "")* as I expected. It is difficult to argue which is the better or more appropriate interpretation; in this context, compatibility with Excel is the issue. An ideal solution could be that the compatibility module inserts parentheses as needed when loading VBA code, to remove ambiguity. Finally, after I added parentheses as needed, the macros successfully generated a proper XML file.

Making the utility for the ITR1 form run on OpenOffice and get an XML output was a challenge, but possible even for a person who knew no BASIC—much less VBA! VBA compatibility in OpenOffice is improving. In all likelihood, by the time we need to file our tax returns next year, we will be able to use OpenOffice with minimal additional effort and generate the XML file for online filing of the return. So, next year, I expect that I will save the Government of India some data entry costs! 

About Anil Seth

The author works as a consultant. Prior to consulting, Anil was a professor at Padre Conceicao College of Engineering (PCCOE) in Goa, managed IT and imaging solutions for Phil Corporation Limited (Goa), supported domestic customers for Tata Burroughs/TIL, and was a researcher with IIT-K and the Indian Institute of Geomagnetism (Mumbai). You can find him online at <http://sethanil.com> and reach him via email at anil@sethanil.com